2022 IEEE International Conference on Automation/XXV Congress of the Chilean Association of Automatic Control (ICA-ACCA) | 978-1-6654-9408-3/22/$31.00 ©2022 IEEE | DOI: 10.1109/ICA-ACCA56767.2022.10006178

# Adversarial Search Algorithms Performance in the Yote Game

Luis I. Torres and Nicolas A. Barriga

***Abstract*—The performance of several adversarial search algorithms is very dependent on the move generation and ordering procedure, as well as on the transposition table characteristics. In this work we compare Alpha-beta, MTD-f and NegaScout performance in the presence and absence of the following features: transposition table, Hash Move heuristic and move ordering using the evaluation function. We found that NegaScout did not yield major improvements over Alpha-Beta, which is likely due to the move ordering performed using a weak evaluation function. In most cases, MTD-f examines fewer nodes, and completes fixed depth searches in less time than Alpha-beta and NegaScout. This is consistent with the literature when using coarse grained evaluation functions. Finally, we found that enabling move ordering leads to fewer nodes examined than enabling the Hash Move heuristic, but, as sorting is time consuming, search is slower. Best results overall are found using MTD-f with all enhancements enabled. These results, in a previously unexplored game, are consistent with existing literature. This enables us to make predictions of algorithm and heuristics' performance in novel games, based on an analysis of the games' features, without having to implement all techniques.**

***Keywords*—Adversarial search, Board games, Game AI, Heuristic search.**

## I. Introduction

SINCE ancient times, there have been board games [1]–[3], where two people faced each other. But, with the advancement of computing, in particular artificial intelligence (AI), the opponent became a machine, which not only aims to play like a human, but also to defeat him. For example, in 1997 Deep Blue's victory over Garry Kasparov in Chess was a crowning achievement for search-based AI. The quest to generate a strategy to defeat the opponent has led to the development of several algorithms that evaluate the possible moves, until finding the one that best suits the player in his turn.

There are many algorithms, which fit the nature of the problem. In this work we will analyze a deterministic, zero-sum and perfect information board game. An example of this type of game is Tic-Tac-Toe. However, this game can be easily solved because its search tree is small compared to games like Chess or Go. Exploring large board games can generate problems of memory or time to process all the information. For this reason, different algorithms have been created to explore as much as possible in an intelligent way [4]. Exploring to greater depths allows the algorithm to know and evaluate a move many turns before it occurs. The most used algorithms to evaluate the best move in this type of games are MiniMax variants: Alpha-beta [5], NegaScout [6], MTD-f [7], NegaC* [8], Best Node Search [9] and SSS* [10].

The moves of a board are evaluated from the current move, which is represented as the root in a search tree, while the possible moves correspond to the nodes or branches of the tree. When a game becomes very large, the best strategy is to avoid exploring nodes that are not good candidates, pruning them from the search tree. The Alpha-beta algorithm picks up the MiniMax/NegaMax idea and avoids exploring nodes that are bad candidates. Also, many positions can be repeated, i.e., a final position could be reached by more than one path. In this case, a transposition table could further reduce exploration.

This work proposes to evaluate the performance of the Alpha-beta, NegaScout and MTD-f algorithms in the Yoté game. The document is organized as follows; in section III, the algorithms used are explained, in section IV the results based on the experiments carried out are presented and finally our conclusions and future work can be found in section V.

## II. Game description

Yoté is a popular african game. It is played on a 6x5 board, similar to a chess board. Each player starts with 12 pieces. Whoever is left with 3 pieces loses the game. The game consists of making moves in turns, each player placing a new piece or moving a piece that is already on the board, to an empty space. Pieces move one space horizontally or vertically, as long as the destination space is empty. If it is occupied by an opponent's piece, it can be jumped and removed from the board (see Figure 1).

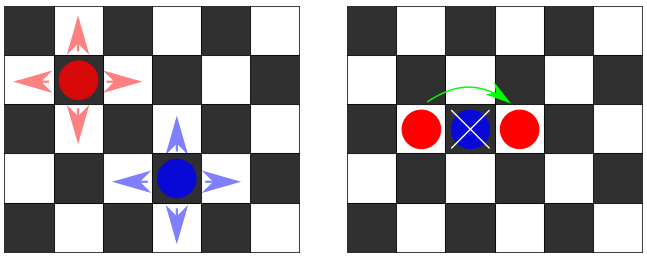


Fig. 1. Valid movements and example of capture. (a) Possible moves of each player and (b) Capture of red player

Luis I. Torres is at the Doctoral Program in Engineering Systems, Faculty of Engineeering, Campus Curicó, University of Talca, Chile. e-mail: luis.torres@utalca.cl

Nicolas A. Barriga is with the School of Videogames Development and VR Engineering, Faculty of Engineering, Campus Talca, University of Talca, Chile. e-mail: nbarriga@utalca.cl

Nicolas A. Barriga is the corresponding author.

978-1-6654-9408-3/22/$31.00 ©2022 IEEE

Authorized licensed use limited to: ANNA UNIVERSITY. Downloaded on August 19,2026 at 14:24:56 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPLEXITIES OF SOMES GAMES.

| Game | Game-tree Complexity | State-space Complexity | Average Branching Factor | Average Game Length |
|---|---|---|---|---|
| Othello | $10^{58}$ | $10^{28}$ | 10 | 58 |
| Chess | $10^{123}$ | $10^{50}$ | 35 | 80 |
| **Yoté** | $\mathbf{10^{158}}$ | $\mathbf{10^{14}}$ | **36** | **101** |
| Go | $10^{360}$ | $10^{172}$ | 250 | 150 |

## III. ALGORITHMS IMPLEMENTED

One of the first techniques devised and that has been the starting point of many others is the MiniMax algorithm. It assumes a player Max, that aims to maximize her score, while the player Min aims to minimize it. The algorithm builds a depth-first tree by starting with the current game state and alternating Max and Min moves. This algorithm solves a large number of problems. The drawback of the algorithm is that is has an exponential time complexity $b^d$, with $b$ being the branching factor, that is, the average number of moves a player can perform in any given state, and $d$ being the depth of the tree—the number of moves until the end of the game. Table I shows the complexity of the Yoté game, along with some other well-known board game for context [11]. On average the length of a Yoté game is 101, which is between Chess (80) and Go (150). On the other hand the average branching factor that was estimated from the Ludii games database is 36 [12]. From this, an upper bound for the game-tree complexity can be estimated to be $36^{101} \approx 10^{158}$ and for the state-space complexity $3^{30} \approx 10^{14}$.

Alpha-beta [5] is an algorithm that prunes nodes from the search tree when it can be proved they can't affect the value of the root. To do this, it maintains an Alpha-Beta window, representing the best and worst values a node can have to have an effect on the root value. If a value is found outside the window, the branch can be safely pruned. With a perfect move ordering, its theoretical complexity is $(\sqrt{b})^d$, which allows it to search twice as deep as MiniMax in the same time. Actual effective branching factor is usually somewhat worse than $\sqrt{b}$, and it depends heavily on move order quality. Since this improvement, new algorithms have continued to be created with the goal of obtaining the greatest amount of information in a reasonable time considering current hardware limitations. One improvement is to perform iterative deepening along with a transposition table, which allows the reuse of information from shallower searches to improve move ordering in deeper searches. The transposition table also allows for several other enhancements, such as the Hash Move heuristic, where the transposition table stores the best move found for each visited node, so it can be tried first when the node is visited in subsequent deeper searches.

NegaScout [6], also called Principal Variation Search, is an algorithm that takes Alpha-Beta's method of pruning branches where a result that can affect the value at the root will not be found, but presents an improvement in its way of pruning. It examines the first successor of a node in the same way as Alpha-Beta, but then all following successors are examined with a null Alpha-Beta window (a so-called scout window), which is faster, but can only prove or disprove if the first child was actually the best. If it fails, it has to repeat the search with the full window. This algorithm's performance is highly dependent on good move ordering, otherwise most scout window searches will fail. In cases where move ordering is not good, Alpha-Beta can be more efficient than NegaScout. Algorithm 1 shows the NegaScout algorithm, where it is assumed that the chosen node represents the best option. To check this assumption a null window is applied (when Alpha and Beta are equal), if this null window fails, it means that the chosen node was not the best one and normal Alpha-Beta pruning should be used.

Another algorithm that takes Alpha-Beta as a basis is MTD-f (Memory-enhanced Test Driver), which is shown in Algorithm 2. It only uses repeated null-window searches. When coupled with a good transposition table it usually outperforms NegaScout. However, it can be sensible to swings in evaluation when switching between even and odd depths, when used with fine-grained evaluation functions.

**Algorithm 1:** NegaScout

```
Function NegaScout(node, depth,α, β, color):
    if node is a terminal node or depth=0 then
        return color x the heuristic value of node
    for each child of node do
        if child is not the first child then
            score ← -NegaScout(child, depth - 1, −α - 1, −α, -color)
            if α <score< β then
                score ← -NegaScout(child, depth - 1, −β, -score, -color)
        else
            score ← -NegaScout(child, depth - 1, −β, −α, -color)
        α ← max(α, score)
        if α ≥ β then
            break
    return α
```

**Algorithm 2:** MTD-f

```
Function MTD-f(root,f,d):
    UpperBound ← +inf
    lowerBound ← −inf
    while lowerBound <UpperBound do
        β←max(f,lowerBound+1)
        f←AlphaBetaWithMemory(root,β-1,β,d);
        if f < β then
            UpperBound ← f
        else
            lowerBound← f
    return f
```

Authorized licensed use limited to: ANNA UNIVERSITY. Downloaded on August 19,2026 at 14:24:56 UTC from IEEE Xplore. Restrictions apply.

TABLE II
AVERAGE BRANCHING FACTOR IN DEPTH 4.
S: SORTING, HM: HASH MOVE, TT: TRANSPOSITION TABLE

| Boards | Opening | Mid-game | End-game |
|---|---|---|---|
| Alpha-beta | 41,2 | - | 43,2 |
| ...+S | 4,5 | 6,2 | 6,8 |
| ...+TT | 41 | - | 39,5 |
| ...+TT+HM | 30,4 | 37,5 | 37,1 |
| ...+S+TT | 3,9 | 5,9 | **4,2** |
| ...+S+TT+HM | 12,2 | 30,1 | 27,4 |
| MTD-f | 29,9 | - | - |
| ...+S | 4,2 | **5,2** | 5,6 |
| ...+TT | 25,4 | - | 40,8 |
| ...+TT+HM | 42,1 | 40,8 | 46,5 |
| ...+S+TT | 13,1 | 11,2 | 5,3 |
| ...+S+TT+HM | 34,3 | 50,3 | 35,4 |
| NegaScout | 38,4 | - | 40 |
| ...+S | 4,2 | 6 | 7,1 |
| ...+TT | 30,5 | - | 41,2 |
| ...+TT+HM | 30 | 36 | 33 |
| ...+S+TT | **3,7** | 5,4 | 4,5 |
| ...+S+TT+HM | 11,8 | 27,7 | 23,4 |

TABLE III
AVERAGE TIME [MS] IN DEPTH 4.
S: SORTING, HM: HASH MOVE, TT: TRANSPOSITION TABLE

| Boards | Opening | Mid-game | End-game |
|---|---|---|---|
| Alpha-beta | 324.091,3 | - | 290.776,3 |
| ...+S | 6.973,7 | 60.998,7 | 47.103,7 |
| ...+TT | 312.537,7 | - | 258.428,3 |
| ...+TT+HM | 1.356,3 | 4.265,0 | 1.037,3 |
| ...+S+TT | 4.071,3 | 16.389,7 | 14.143,0 |
| ...+S+TT+HM | **1.220,0** | **3.460,3** | **864,3** |
| MTD-f | 156.079,7 | - | - |
| ...+S | 7.598,3 | 45.014,0 | 92.620,3 |
| ...+TT | 73.973,7 | - | 75.267,0 |
| ...+TT+HM | 533,7 | 1.516,3 | 660,0 |
| ...+S+TT | 486,3 | 4.260,7 | 4.394,3 |
| ...+S+TT+HM | **200,7** | **734,0** | **397,0** |
| NegaScout | 271.562,0 | - | 250.944,7 |
| ...+S | 8.197,0 | 43.286,0 | 34.715,3 |
| ...+TT | 136.090,7 | - | 177.041,0 |
| ...+TT+HM | 1.355,0 | 2.328,7 | 1.080,3 |
| ...+S+TT | 4.377,0 | 10.624,3 | 8.701,3 |
| ...+S+TT+HM | **1.975,3** | **2.070,3** | **1.297,7** |

TABLE IV
AVERAGE NODES EXPANDED IN DEPTH 4.
S: SORTING, HM: HASH MOVE, TT: TRANSPOSITION TABLE

| Boards | Opening | Mid-game | End-game |
|---|---|---|---|
| Alpha-beta | 9.819.737,3 | - | 5.985.120,0 |
| ...+S | 16.551,3 | 56.232,7 | 81.048,7 |
| ...+TT | 9.599.751,3 | - | 4.973.551,0 |
| ...+TT+HM | 25.976,0 | 20.320,3 | 13.066,0 |
| ...+S+TT | 8.344,0 | 11.637,0 | 16.818,7 |
| ...+S+TT+HM | **8.294,7** | **11.533,7** | **7153,3** |
| MTD-f | 3.106.140,3 | - | - |
| ...+S | 15.340,7 | 46.021,7 | 141.927,3 |
| ...+TT | 1.189.411,3 | - | 1.603.986,0 |
| ...+TT+HM | 12.683,7 | 12.253,0 | 8.467,7 |
| ...+S+TT | 3.307,3 | 4.755,3 | 5.907,0 |
| ...+S+TT+HM | **3.190,0** | **4.764,0** | **3.863,3** |
| NegaScout | 7.520.508,3 | - | 4.947.241,0 |
| ...+S | 15.326,7 | 54.413,7 | 69.371,7 |
| ...+TT | 2.726.218,7 | - | 4.188.390,3 |
| ...+TT+HM | 25.569,3 | 19.421,0 | 11.960,3 |
| ...+S+TT | 7.982,7 | 10.980,0 | 9.848,0 |
| ...+S+TT+HM | **7.933,3** | **10.876,0** | **5.869,0** |

TABLE V
AVERAGE DEPTH IN 30 [S].

| Boards | Opening | Mid-game | End-game |
|---|---|---|---|
| Alpha-beta | 4,7 | 3 | 4 |
| ...+S | 5,3 | 4 | 4,3 |
| ...+TT | 4,7 | 3 | 4,3 |
| ...+TT+HM | 6,7 | 5,7 | 6,7 |
| ...+S+TT | 6,3 | 4,7 | 5 |
| ...+S+TT+HM | 6,7 | 6 | **7** |
| MTD-f | 4,7 | 3,3 | 3,7 |
| ...+S | 5,7 | 4 | 4,7 |
| ...+TT | 5 | 4 | 4,7 |
| ...+TT+HM | 7 | 6,3 | 7 |
| ...+S+TT | 7 | 4,7 | 5,3 |
| ...+S+TT+HM | **7,3** | **6,7** | **7** |
| NegaScout | 4,7 | 3,3 | 4,3 |
| ...+S | 5,3 | 4 | 4,3 |
| ...+TT | 4,7 | 3,3 | 4,7 |
| ...+TT+HM | 6,7 | 5,7 | 6,3 |
| ...+S+TT | 6,3 | 5 | 5,3 |
| ...+S+TT+HM | 6,7 | 6 | 6,7 |

## IV. EXPERIMENTAL RESULTS

For all experiments we used a PC with Intel Core i5 CPU 8250U processor at 1.6GHz, with 8GB of RAM, running Windows 10. All all algorithms were implemented in Python 3.8.11. The experiments performed have been limited to a maximum depth of 8 at a maximum time of 30 seconds per depth. However, time does not stop at 30 seconds if it is exceeded, but will let it finish evaluating at the depth in which it is located and thus end the evaluation of the algorithm. It is important to take into account this consideration for the results presented in the time table III. On the other hand, the proposed study has considered the evaluation of 9 game boards, which 3 are in an opening game, 3 in a mid-game and 3 in an end-game. From these variants the average in each algorithm has been obtained. In all cases, improvements have been made to the algorithms to increase their performance. Alpha-beta, NegaScout and MTD-f, as well as a transposition table (TT) [13] have been implemented, to which 2 improvements were added. First, an ordering of the successors (S) [14] was performed, which is a requirement for a good performance in NegaScout. Second, the Hash Move (HM) heuristic [15], [16] was added to the transposition table to improve speed and thus obtain a good performance in the MTD-f algorithm. These improvements were activated and deactivated to evaluate the algorithms with and without these improvements. The performance study has been considering the average branching factor, the time taken to find a solution,

Authorized licensed use limited to: ANNA UNIVERSITY. Downloaded on August 19,2026 at 14:24:56 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
PERFORMANCE OF TT WITH S AND HM IN DEPTH 4

| | Opening boards | | | Mid-game boards | | | End-game boards | | |
|---|---|---|---|---|---|---|---|---|---|
| | Explored Nodes | Alpha-beta breaks | TT uses | Explored Nodes | Alpha-beta breaks | TT uses | Explored Nodes | Alpha-beta breaks | TT uses |
| Alpha-beta | 8.294,7 | 584,3 | 1.519,0 | 11.533,7 | 362,7 | 2.020,7 | 7.153,3 | 211,0 | 1.292,0 |
| MTD-f | **3.190,0** | **63,0** | **146,0** | **4.764,0** | **115,3** | **778,3** | **3.863,3** | **72,3** | **543,3** |
| NegaScout | 7.933,3 | 588,3 | 1.519,0 | 10.876,0 | 366,0 | 2.000,3 | 5.869,0 | 227,7 | 1.294,3 |

the nodes explored, the speed and the average depth reached.

Table II shows the average branching factors when using different improvements in the algorithms. In an opening board, NegaScout with the successors ordered in the transposition table performs the best, MTD-f performs the best when the boards are in mid-games and Alpha-beta with the transposition table ordered when the games are about to end.

The Table III shows the time in milliseconds required to perform a exploration up to depth 4. In the initial stage of the game, all algorithms were able to reach minimum depth 4. However, in the other two stages, not all algorithms were able to finish within the allotted time, which has been marked with a dash. In the cases of Alpha-beta, MTD-f and NegaScout, the best times in all stages occurred when the transposition table had the successor ordering and the Hash Move implemented. This improvement is also reflected in the number of nodes checked (see Table IV), which by having these improvements activated saves memory in the number of nodes explored.

The tables presented have shown the performances up to depth 4, which could be reached in most cases by all the algorithms with and without the transposition table. However, by applying the improvements in the algorithms and considering the limitation of 30 seconds in the exploration of each depth, many of these implementations were able to overcome this depth. Table V shows the averages achieved, where MTD-f with a transposition table with ordering and Hash move achieved the best performance in all the boards, however in the end-games, it reaches the same performance as an Alpha-beta with the same characteristics in its transposition table. On the other hand, when using a depth limit of 4 (see Table VI) it can be observed that MTD-f is the one with the best performance compared to all the studied algorithms.

## V. CONCLUSIONS

We have implemented several standard algorithms and enhancements for adversarial search and compared their performance in the game of Yoté.

We found that the MTD-f algorithm is the best performing algorithm in the Yoté game when using a transposition table, Hash Move heuristic and move ordering. This is consistent with previous research. According to the literature, NegaScout only needs its successors sorted to perform correctly. However, sorting is computationally expensive, so relying on the Hash Move shows better results.

### A. Future Work

Experiments across different games would be of use, to be able to identify game characteristics which affect the performance of each algorithm and enhancement. Comparing to newer algorithms, such as NegaC*, could also lead to insights on their relative performance. Additionally, another interesting algorithm to apply to this game is the one introduced by Silver et al. [17], which uses Monte-Carlo Tree Search, with Deep Convolutional Neural Networks for prior probabilities and leaf evaluations. It has been successfully applied to several board games [18].

## REFERENCES


[1] L. Kurke, "Ancient greek board games and how to play them," *Classical Philology*, 1999.

[2] K. Masukawa, "The origins of board games and ancient game boards," in *Simulation and Gaming in the Network Society*, 2016.

[3] H. Morris, "International perspectives: Universal games from a to z," *Mathematics Teaching in the Middle School MTMS*, vol. 5, no. 5, pp. 300 – 305, 2000.

[4] L. Tommy, M. Hardjianto, and N. Agani, "The analysis of alpha beta pruning and MTD(f) algorithm to determine the best algorithm to be implemented at connect four prototype," *IOP Conference Series: Materials Science and Engineering*, vol. 190, p. 012044, apr 2017.

[5] J. Pearl, "The solution for the branching factor of the alpha-beta pruning algorithm and its optimality," *Commun. ACM*, vol. 25, no. 8, p. 559–564, Aug. 1982.

[6] A. Reinefeld, "An improvement to the scout tree search algorithm," *ICCA journal*, 1983.

[7] E. Stock and D. King, "A new enhancement to mtd(f)," in *15th International Computer Games Conference*, 2010, 15th International Computer Games Conference, CGAMES 2010 ; Conference date: 28-07-2010 Through 31-07-2010.

[8] J.-C. Weill, "Experiments with the negac* search an alternative for othello endgame search," *Heuristic Programming in Artificial Intelligence, the second computer olympiad*, pp. 174–188, 1991.

[9] D. Rutko, "Fuzzified algorithm for game tree search with statistical and analytical evaluation," *Datorzinātne un informācijas tehnologìjas*, p. 90, 2011.

[10] U. Lorenz and T. Tscheuschner, "Player modeling, search algorithms and strategies in multi-player games," in *Advances in Computer Games*, H. J. van den Herik, S.-C. Hsu, T.-s. Hsu, and H. H. L. M. J. Donkers, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 210–224.

[11] L. V. Allis, "Searching for solutions in games and artificial intelligence," Ph.D. dissertation, Ph.D. dissertation, University of Limburg, Maastricht, 1994.

[12] C. Browne. (2021) Ludii general game. [Online]. Available: https://ludii.games/details.php?keyword=Yote

[13] J. Veness and A. Blair, "Effective use of transposition tables in stochastic game tree search," in *2007 IEEE Symposium on Computational Intelligence and Games*, 2007, pp. 112–116.

[14] Z. Qi, X. Huang, Y. Shen, and J. Shi, "Optimization of connect6 based on principal variation search and transposition tables algorithms," in *2020 Chinese Control And Decision Conference (CCDC)*, 2020, pp. 198–203.

[15] R. Hyatt and A. Cozzie, "The effect of hash signature collisions in a chess program," *J. Int. Comput. Games Assoc.*, vol. 28, pp. 131–139, 2005.

[16] L. Huizhan, X. Chenjun, L. Hongye, and W. Jiao, "Hash table in chinese chess," *2012 24th Chinese Control and Decision Conference (CCDC)*, pp. 3286–3291, 2012.


Authorized licensed use limited to: ANNA UNIVERSITY. Downloaded on August 19,2026 at 14:24:56 UTC from IEEE Xplore. Restrictions apply.

[17] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis, "Mastering the game of Go with deep neural networks and tree search," *Nature*, vol. 529, no. 7587, pp. 484–489, 2016. [Online]. Available: https://doi.org/10.1038/nature16961

[18] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel *et al.*, "A general reinforcement learning algorithm that masters chess, shogi, and go through self-play," *Science*, vol. 362, no. 6419, pp. 1140–1144, 2018.

Authorized licensed use limited to: ANNA UNIVERSITY. Downloaded on August 19,2026 at 14:24:56 UTC from IEEE Xplore. Restrictions apply.